

RESOLUCIÓN DE PROBLEMAS Y ALGORITMOS

Dra. Jessica Andrea Carballido

jac@cs.uns.edu.ar

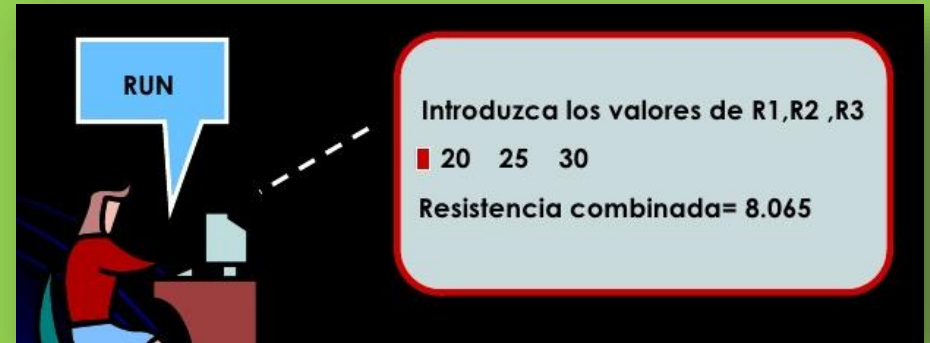


Dpto. de Ciencias e Ingeniería de la Computación

UNIVERSIDAD NACIONAL DEL SUR

VISION ESTATICA $\neq$ VISION DINAMICA

```
01: Var
02:   a,b: array [1..1000] of char;
03:
04: Function max(a,b: longint): longint;
05:   begin
06:     if (a> b) then max:= a
07:       else max:= b;
08:   end;
09:
10: Function LCS(m,n: longint): longint;
11:   var i,j: longint;
12:     x,y: array [0..1000] of longint;
13:   begin
14:     fillchar(y, sizeof(y), 0);
15:     for i:= 1 to m do
16:       begin
17:         x:= y;
18:         for j:= 1 to n do
19:           if (a[i]= b[j]) then y[j]:= x[j-1]+1
20:             else y[j]:= max(x[j], y[j-1]);
21:         end;
22:       LCS:= y[n];
23:     end;
24:
25: Begin
26:   while (not eof(input)) do
27:     begin
28:       readln(input,a);
29:       readln(input,b);
30:       writeln(output,LCS(length(a), length(b)));
31:     end;
32: End.
```



Ejecutar,
simular la ejecución
haciendo una traza

Iteración



La estructura de control ITERATIVA permite modelar problemas en los cuales una **secuencia de instrucciones** debe ejecutarse varias veces.

La cantidad de iteraciones puede quedar determinada por una **variable de control** que toma valores dentro de un rango escalar o por una **expresión lógica**.

En el primer caso hablamos de iteración con **contador** y en el segundo hablamos de iteración con **condición**.

Iteración

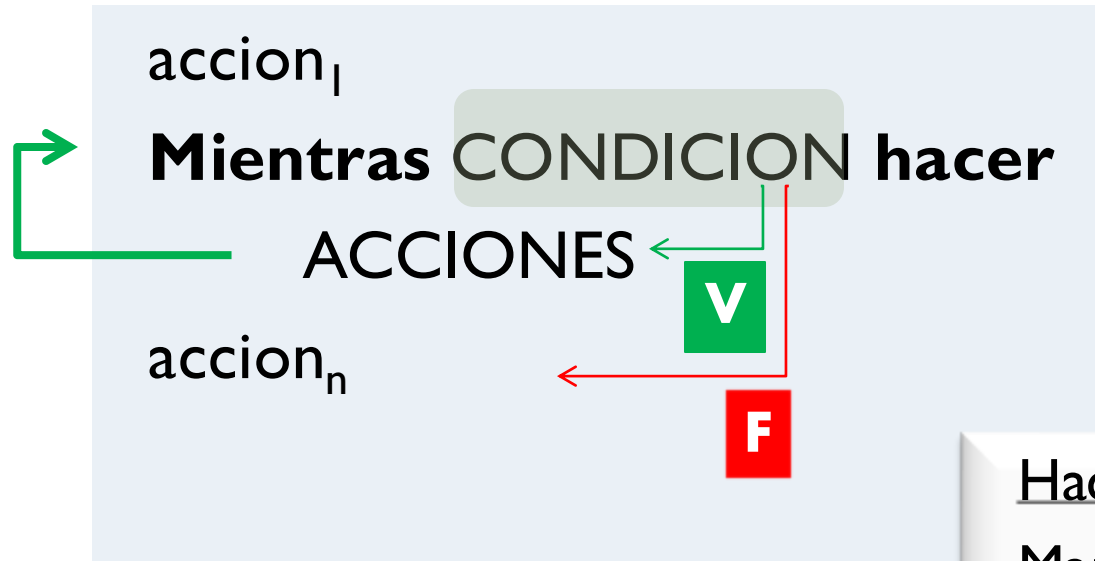


ITERACION CON CONDICION

while

repeat

Iteración con condición



Existe la posibilidad de
que nunca se ejecuten
las acciones del
“mientras”

Hacer una llamada telefónica

Marcar el número

Mientras teléfono ocupado
hacer

Colgar

Marcar el número

Esperar a que atiendan

Hablar

Iteración con condición



La **sentencia** (ya sea simple o compuesta)
del *while* es el bloque iterativo
(es decir, es lo que se repite).



Cuando la expresión lógica es falsa
se produce la condición de corte de la iteración.

En el bloque iterativo debe modificarse
al menos una de las variables de la expresión lógica,
para provocar que la expresión lógica se evalúe con valor falso
y el bucle termine.

Ejemplo
NAFTA

Iteración con condición



```
while expresión lógica do  
    sentencia simple o compuesta
```

- La sentencia simple o compuesta es el **bloque iterativo**, y puede no ejecutarse nunca en un caso donde desde el inicio, la expresión lógica sea FALSA.
- Si la condición de corte no se verifica nunca, es decir la expresión lógica nunca da como resultado FALSO (siempre es VERDADERA), se produce un **bucle infinito**.



```
A:=1;  
While (A<10) do  
    Write(A);
```

Iteración con condición

Problema: Mostrar los números del **N hasta el 1**, a partir de un N natural que es ingresado por el usuario.

Algoritmo MostrarNums

DE: N (natural)

DS: -

Mientras (**$N > 0$**)

Mostrar (N)

$N \leftarrow N - 1$



Ingrese un nro. natural:

5

Respuesta:

5 4 3 2 1

¿Qué pasa si ingresan un 0?
O un número negativo.

Pasar a
Pascal

En el bloque iterativo debe modificarse
al menos una de las variables de la expresión lógica,
para provocar que la expresión lógica se evalúe con valor falso
y el bucle termine.

Iteración con condición

Problema: Mostrar los números del 1 hasta el N, a partir de un N natural que es ingresado por el usuario.



Algoritmo MostrarNums

DE: N (natural)

DS: -

Daux: aux

$\text{aux} \leftarrow 1$

Mientras ($\text{aux} \leq N$)

Mostrar (aux)

$\text{aux} \leftarrow \text{aux} + 1$

Ingrese un nro. natural:
8
Respuesta:
1 2 3 4 5 6 7 8

¿Qué pasa si ingresan un 0?
O un número negativo,
o un 1...

Pasar
a Pascal

Iteración con condición

Problema: Contar la cantidad de dígitos de un número N.



Handwritten work on a piece of paper:

$N = 1278$ (with a red arrow pointing to the 8 and the text "Div 10" written in red)

Division:

$$\begin{array}{r} 1278 \quad | \quad 10 \\ \underline{127} \\ 8 \end{array}$$

Result of division:

$$N \text{ div } 10 = 127$$
$$N \text{ mod } 10 = 8$$



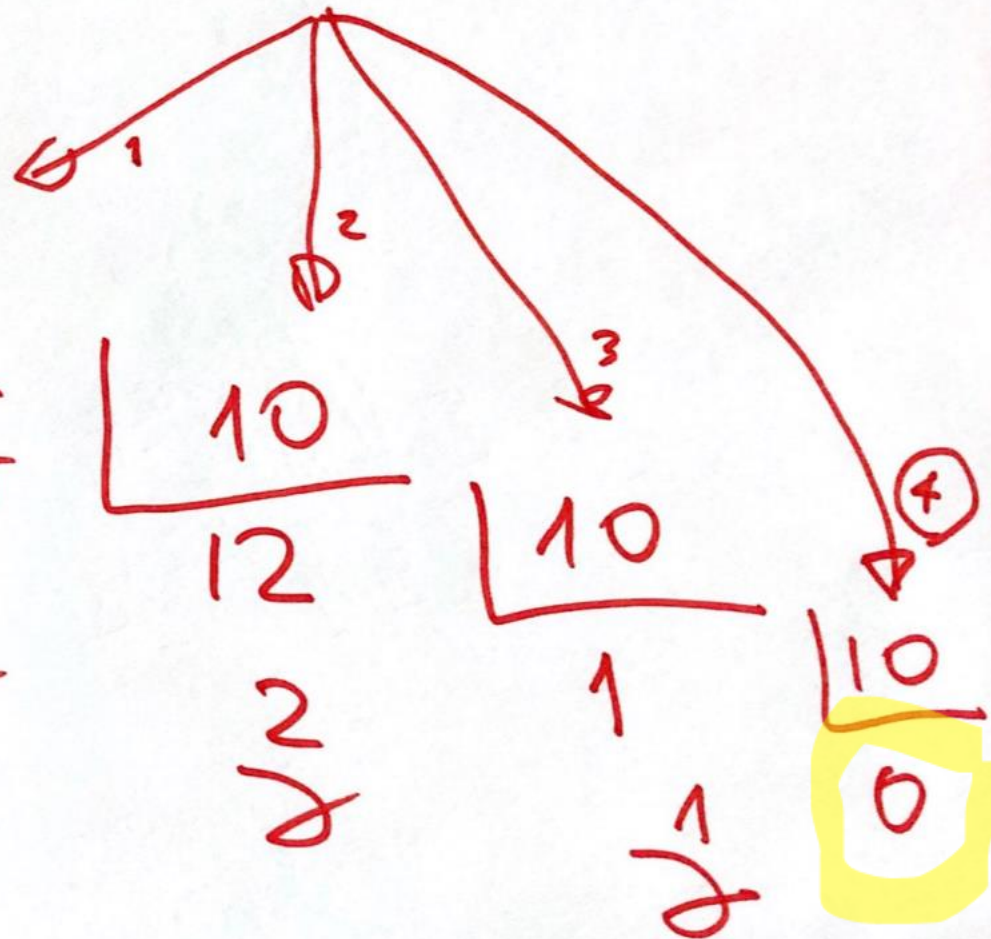
Iteración con condición

Problema: Contar la cantidad de dígitos de un número N.

$N = 1278$

1278
8
8

1278
10
127
7
8



Iteración con condición

Problema: Contar la cantidad de dígitos de un número N.



Algoritmo CantDigitos

DE: N (entero)

DS: cant (entero)

$\text{cant} \leftarrow 0$

Mientras (N tiene dígitos)

 sacar un dígito de N

$\text{cant} \leftarrow \text{cant} + 1$

Nuevo concepto:
ACUMULADOR
(cant)

Iteración con condición



Problema: Contar la cantidad de dígitos de un número N .

Algoritmo CantDigitos

DE: N (entero)

DS: cant (entero)

cant $\leftarrow 0$

Mientras (N tiene dígitos)

sacar un dígito de N

cant $\leftarrow$ cant+1

($N > 0$)

$N \leftarrow N \text{ div } 10$

En el bloque iterativo debe modificarse
al menos una de las variables de la expresión lógica,
para provocar que la expresión lógica se evalúe con valor falso
y el bucle termine.

Iteración con condición

Problema: Contar la cantidad de dígitos de un número N.

Algoritmo CantDigitos

DE: N (entero)

DS: Cant (entero)

cant $\leftarrow$ 0

Mientras (N tiene dígitos)

 sacar último dígito de N

 cant $\leftarrow$ cant+1



La condición del “mientras”
es que N tenga dígitos.
En las acciones voy
eliminando dígitos de N,
luego en algún momento
N no tendrá más dígitos
(condición de corte).

En el bloque iterativo debe modificarse
al menos una de las variables de la expresión lógica,
para provocar que la expresión lógica se evalúe con valor falso
y el bucle termine.

Iteración con condición

Problema: Contar la cantidad de dígitos de un número N.



Algoritmo CantDigitos

DE: N (entero)

DS: Cant (entero)

cant $\leftarrow$ 0

Mientras ($N > 0$)

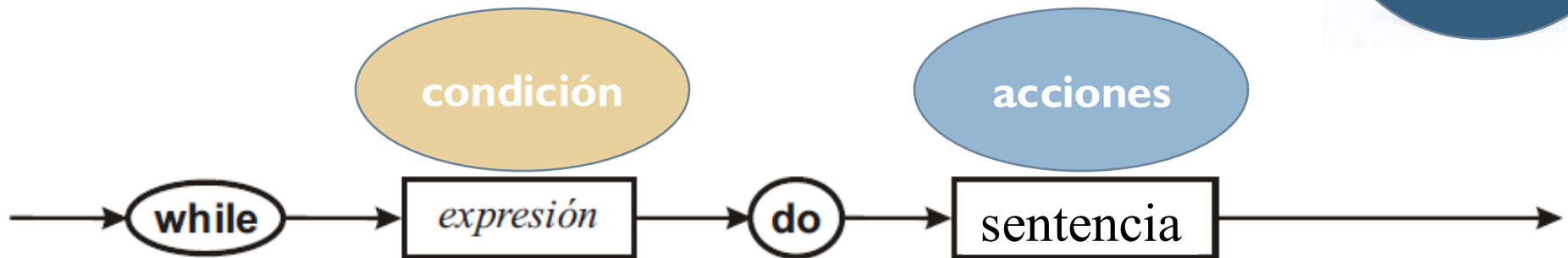
$N \leftarrow N \text{ div } 10$
cant $\leftarrow$ cant + 1

Podríamos pedir la cantidad de dígitos
de un número negativo?
Y si ingresan $N=0$?

En el bloque iterativo debe modificarse
al menos una de las variables de la expresión lógica,
para provocar que la expresión lógica se evalúe con valor falso
y el bucle termine.

Diagrama de SINTAXIS:WHILE

Visión
estática

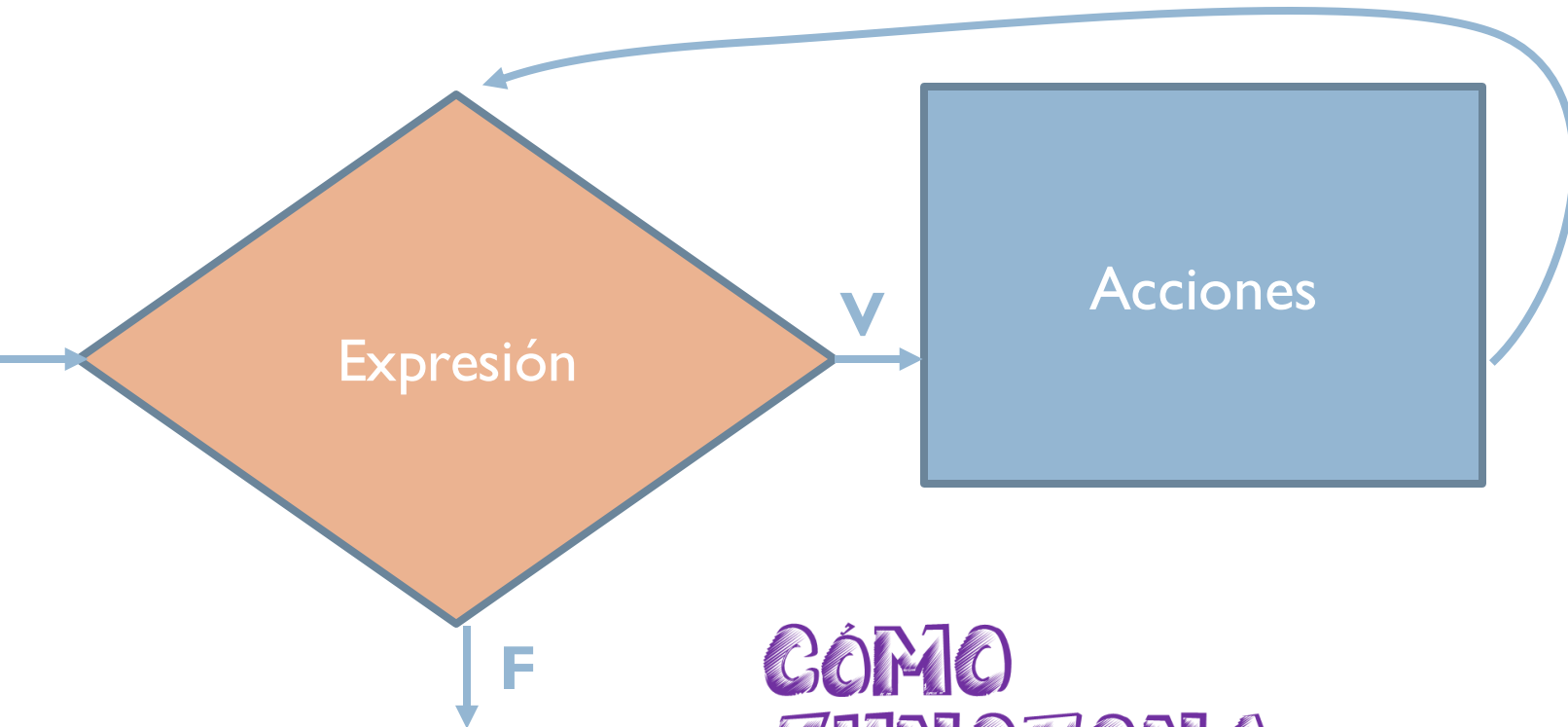


CÓMO SE ESCRIBE

Diagrama de flujo: WHILE

Visión
dinámica

Qué pasa en ejecución



**CÓMO
FUNCIONA**

Iteración con condición



Problema: Calcular la suma de los dígitos de un número entero positivo N .

Por ejemplo si $N = 605$ la suma es 11.

Usamos un acumulador de sumas:

0+5 (d_1 o sea dígito en la posición 1 de N)
... +0 (d_2)
... +6 (d_3)

Estrategia para la resolución:

Acumulo el último dígito de N , lo elimino,
acumulo el último dígito de N , lo elimino,
así hasta que N no tenga más dígitos.

Iteración con condición



Algoritmo SumaDigitos

DE: N

DS: suma

Comienzo

suma $\leftarrow$ 0

Mientras (N>0) hacer

 suma $\leftarrow$ suma + N mod 10

 N $\leftarrow$ N div 10

Fin

Operaciones básicas:

N mod 10 (**expresión** para **consultar**
el último dígito de N)

N:=N div 10 (**acción** para **eliminar** el último dígito de N)

{Procesamiento}

 suma := 0;

 while (N > 0) do

 begin

 suma := suma + N mod 10;

 N := N div 10;

 end;



Realizar una traza para dos casos de prueba.

Iteración con condición

```
Program sumaDigitos;  
Var suma, N: integer;  
Begin  
  {Lectura de datos de entrada}  
  Writeln('Ingrese un valor para N:');  
  Readln(N);  
  {Procesamiento}  
  suma := 0;  
  while (N > 0) do  
  begin  
    suma := suma + N mod 10;  
    N := N div 10;  
  end;  
  {Mostrar resultado}  
  Writeln('La suma es:', suma);  
end.
```



Iteración con condición



Problema: Calcular la suma de los dígitos **impares** de un número natural N .

Por ejemplo

- Si $N=605$, la suma de sus dígitos impares es 5.
- Si $N=1211$, la suma de sus dígitos impares es 3.
- Si $N=32517$, la suma de sus dígitos impares es 16.
- Si $N=42$, la suma es 0.



Iteración con condición

```
Program OJO;  
Var N, suma: Integer;  
Begin  
  read(N);  
  suma:=0;  
  while (N>0) do  
    begin  
      if (N mod 2 <> 0) // si N es impar, el ultimo dígito de N es impar  
      then  
        begin  
          suma := suma + N mod 10; // agrego el último dígito a la bolsa de sumas  
          N := N div 10; // lo saco de N  
        end;  
      end;  
    write(suma);  
  End.
```



Hacer traza

Iteración con condición

```
Program OJO;  
Var N, suma: Integer;  
Begin  
  read(N);  
  suma:=0;  
  while (N>0) do  
    begin  
      if (N mod 2 <> 0) // si N es impar, el ultimo dígito de N es impar  
      then  
        begin  
          suma := suma + N mod 10; // agrego el último dígito a la bolsa de sumas  
          N := N div 10; // lo saco de N SI ES IMPAR!!  
        end;  
      end;  
    write(suma);  
  End.
```



Hacer traza

```
Program esteAnda;  
Var N, suma: Integer;  
Begin  
  read(N);  
  suma:=0;  
  while (N>0) do  
    begin  
      if (N mod 2 <> 0) // si N es impar, el ultimo dígito de N es impar  
        then  
          begin  
            suma := suma + N mod 10; // agrego el último dígito a la bolsa de sumas  
            N := N div 10; // lo saco de N  
          end  
        else  
          N := N div 10; // lo tengo que sacar también si es par  
        end;  
      write(suma);  
    End.
```


Iteración con condición

```
Program esteAnda2;  
Var N, suma: Integer;  
Begin  
  read(N);  
  suma:=0;  
  while (N>0) do  
    begin  
      if (N mod 2 <> 0) // si N es impar, el ultimo dígito de N es impar  
        then  
          suma := suma + N mod 10; // agrego el último dígito a la bolsa de sumas  
          N := N div 10; // lo saco de N POR FUERA DEL IF!!  
        end;  
      write(suma);  
    end;  
  End.
```

Hacer traza

Iteración con condición



Problema: Calcular el promedio de los dígitos impares de un número natural N.

Por ejemplo

- Si $N=605$, el promedio de sus dígitos impares es $5 = (5/1)$.
- Si $N=1211$, el promedio de sus dígitos impares es $1 = (1+1+1)/3$.
- Si $N=32517$, el promedio de sus dígitos impares es $4 = (7+1+5+3)/4$.
- Si $N=42$, se debe indicar que N no tiene dígitos impares.

```
program promIMPARES;
var N, suma, cant: integer;
begin
  writeln('Ingrese el número:'); readln(N);
  cant:=0; suma:=0;
  while (N>0) do
  begin
    if (N mod 2 <> 0) {si el num es impar, el ult dígito es impar}
    then
      begin
        suma:=suma + N mod 10;
        cant:=cant+ 1;
      end;
    N:=N div 10;
  end;
  if (cant=0) then writeln('El numero no tiene dígitos impares')
  else writeln('El promedio de los dig impares es:', suma/cant);
end.
```

Fin intro WHILE

Iteración con condición



Repetir
ACCIONES
hasta CONDICION

Expresión Lógica

ACCIONES
Si CONDICION = FALSA
Entonces
 ACCIONES
Si CONDICION = FALSA
Entonces
 ACCIONES
...
Si CONDICION = VERDADERA
Entonces
 FIN

Iteración con condición



accion₁

Repetir

ACCIONES

hasta CONDICION

accion_n



F

V



Al menos **UNA VEZ**
voy a batir las claras
un minuto.

Hacer merengue de chocolate

Incorporar 2 claras

Repetir

Batir las claras un minuto

Hasta batido a punto letra

Incorporar el cacao



Iteración con condición



```
repeat  
  sentencia  
until expresión lógica
```

- La sentencia es el **bloque iterativo** y se ejecuta por lo menos una vez.
- Cuando la expresión lógica computa **verdadero** se produce la **condición de corte**.
- El bloque iterativo debe modificar al menos una de las variables involucradas en la expresión lógica para garantizar que la condición de corte se produzca.
- Si la condición de corte no se verifica nunca, se produce un **bucle infinito**.

Iteración con condición

Problema: Mostrar los números del **N hasta el 1**, a partir de un N natural que es ingresado por el usuario.

Algoritmo MostrarNums

DE: N (natural)

DS: -

Mientras (**N > 0**)

Mostrar (N)

N ← N - 1

Ingrese un nro. natural:

5

Respuesta:

5 4 3 2 1

Qué pasa si ingresan un 0?

O un número negativo.

Iteración con condición

Problema: Mostrar los números del **N** hasta el **1**, a partir de un N natural que es ingresado por el usuario.



```
1
2 program mostrar;
3
4 var N: integer; // Validar que sea un número natural.
5
6 begin
7 // Ingreso de N, validando que solo aceptaremos un número natural.
8 repeat
9     writeln('Ingrese un número natural. ');
10    read(N);
11 until (N>0);
12
13 // Mostramos desde N hasta 1
14 while (N>0) do
15     begin
16         write(N);
17         N:=N-1;
18     end;
19
20 end.
```

```
20 lines compiled, 0.1 sec
Ingrese un número natural.
5
54321
```

```
20 lines compiled, 0.0 sec
Ingrese un número natural.
0
Ingrese un número natural.
-8
Ingrese un número natural.
-5
Ingrese un número natural.
4
4321
```

¡Se puede utilizar para validar el ingreso de los datos!



Validar dos números a y b, mayores a 0, con b mayor que a.

repeat

writeln('Ingrese un valor positivo para a:');

readln(a)

until (a>0);

repeat

writeln('Ingrese un valor para b (mayor que a):');

readln(b)

until (b>a);

¡Se puede utilizar para validar el ingreso de los datos!

Validar el ingreso de los siguientes datos:

- Un número entero que sea positivo (mayor de cero).
- Un número entero que sea positivo y par.
- Un número entero que sea un único dígito.
- Un caracter que sea letra.
- Un caracter que sea vocal mayúscula.



```
var n: integer;  
begin
```

```
...
```

```
repeat
```

```
    writeln('Ingrese un número mayor que cero: ');
```

```
    readln(n)
```

```
until (n>0);
```

```
...
```

Un número entero que sea positivo (distinto de cero).

```
var n: integer;  
begin
```

```
...
```

```
repeat
```

```
    writeln('Ingrese un número PAR mayor que cero: ');
```

```
    readln(n)
```

```
until ((n>0) and (n mod 2=0));
```

```
...
```

Un número entero que sea positivo (distinto de cero) y par.

```
var n: integer;  
begin
```

```
...
```

```
repeat
```

```
    writeln('Ingrese un dígito: ');
```

```
    readln(n)
```

```
until ((n>=0) and (n<=9));
```

```
...
```

Un número entero que sea un DÍGITO.

```
var ch: char;  
begin
```

```
...
```

```
repeat
```

```
    writeln('Ingrese una letra: ');
```

```
    readln(ch)
```

```
until ((ch>='a') and (ch<='z')) or ((ch>='A') and (ch<='Z'));
```

```
...
```

Un caracter que sea una letra.

```
var ch: char;
```

```
begin
```

```
...
```

```
repeat
```

```
    writeln('Ingrese una vocal mayúscula: ');
```

```
    readln(ch)
```

```
until ((ch='A') or (ch='E') or (ch='I') or (ch='O') or (ch='U'));
```

```
...
```

Un caracter que sea una vocal MAYUSCULA.

Buscar: diagrama de sintaxis y diagrama de flujo de REPEAT.

Fin intro REPEAT



Iteración



ITERACION CON CONTADOR

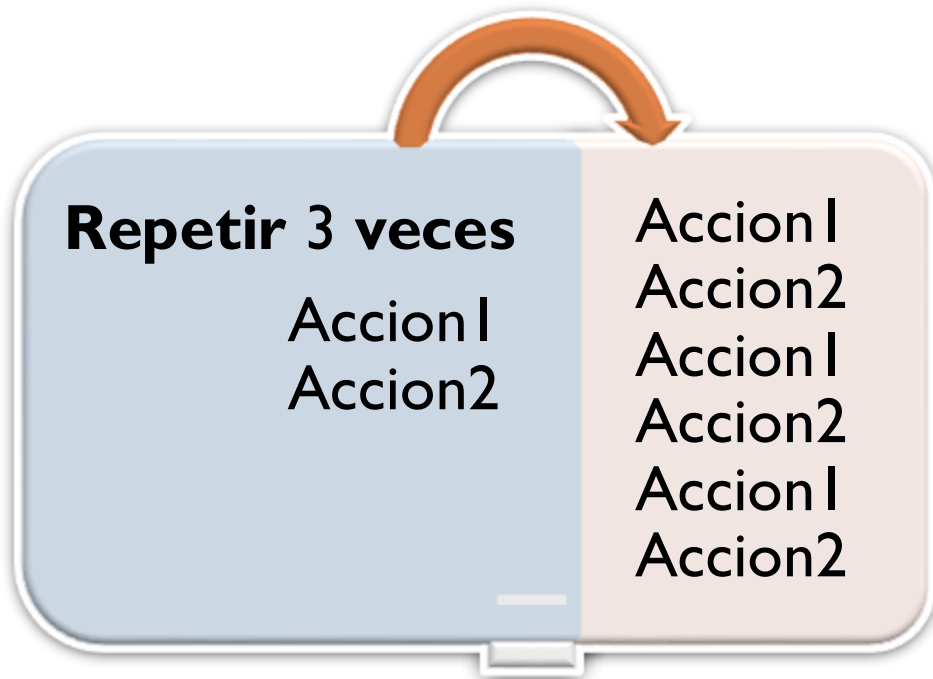
for

Iteración con contador



Repetir N veces
ACCIONES

Número natural



Ejercicio de Pilates

Recostarse en la colchoneta

Repetir 30 veces

Elevar las piernas

Bajarlas lentamente

Elongar

Iteración con contador



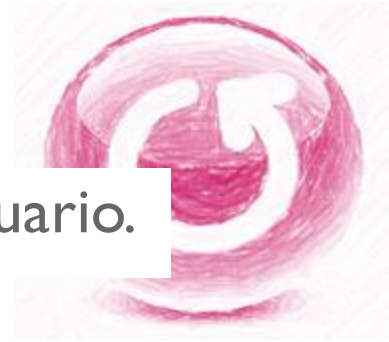
**Repetir N veces
ACCIONES**

**Número
natural**

**PARA i desde: I hasta: N HACER
ACCIONES**

Iteración con condición

Problema: Mostrar un '*' tantas veces como lo indique el usuario.



REPETIR N VECES
mostrar *

i es la variable de
control (puede
tener cualq
nombre)

Algoritmo ASTERISCOS

DE: N

DS: Asteriscos mostrados en pantalla

Daux: i

Para i desde 1 hasta N hacer
Mostrar('*')

Iteración con contador

Problema: Mostrar un '*' tantas veces como lo indique el usuario.



Algoritmo ASTERISCOS

DE: N

DS: *(Asteriscos mostrados en pantalla)*

Daux: i

Para i desde 1 hasta N hacer
 Mostrar('*')

Program Asteriscos;

var i, N : integer;

begin

writeln('Ingrese la cantidad de *');

readln(N);

for $i := 1$ to N do

 write('*');

end.

Iteración con contador



Problema: Mostrar todos los números naturales desde 1 hasta N, con N natural ingresado por el usuario.

Algoritmo Naturales

DE: $\mathbb{N}$

DS: *(Numeros mostrados en pantalla)*

Daux: i

Para i desde 1 hasta N hacer
 Mostrar(i)

Program Naturales;

var i, N : integer;

begin

writeln('Ingrese N:');

readln(N);

for $i := 1$ to N do

 write(i);

end.

Iteración con contador

Problema: Mostrar todos los números naturales desde 1 hasta N, con N natural ingresado por el usuario.



```
Program Naturales;  
var aux, N: integer;  
begin  
  writeln('Ingrese N:');  
  readln(N);  
  aux:=1;  
  while (aux<=N) do  
  begin  
    write(aux);  
    aux:=aux+1;  
  end;  
end.
```

```
Program Naturales;  
var aux, N: integer;  
begin  
  writeln('Ingrese N:');  
  readln(N);  
  for aux:=1 to N do  
    write(aux);  
  end.
```

Iteración con contador

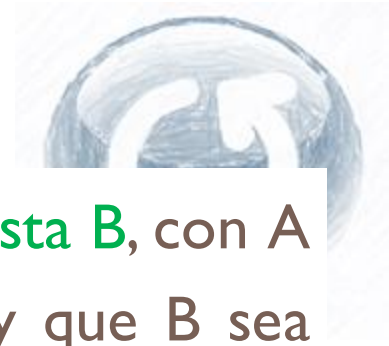
Problema: Mostrar todos los números naturales desde N hasta 1, con N natural ingresado por el usuario.



```
Program Naturales;  
var N: integer;  
begin  
  writeln('Ingrese N:');  
  readln(N);  
  while (N>0) do  
  begin  
    write(N);  
    N:=N-1;  
  end;  
end.
```

```
Program Naturales;  
var i, N: integer;  
begin  
  writeln('Ingrese N:');  
  readln(N);  
  for i:=N downto 1 do  
    write(i);  
  end.
```

Iteración con contador



Problema: Mostrar todos los números naturales desde A hasta B, con A y B ingresados por el usuario. Validar que A sea positivo y que B sea mayor que A.

Pueden mandar la solución por mail

Iteración con contador



Problema: A partir de una base y un exponente
calcular la potencia ($\text{base}^{\text{exponente}}$)

Ej.: base=2, exponente=4 debe calcular 2^4

¿Cuáles son los datos de entrada?

¿Cuáles son los datos de salida?

¿Cuáles pueden ser los casos de prueba?

Si base=2, exponente=4 debo calcular $2*2*2*2$

Acumulación de **productos** sucesivos:

Exponente veces	1	*	2			
	..		*	2		
	..			*	2	
	..				*	2

Iteración con contador

Algoritmo Potencia

DE: base, exponente

DS: pot

Daux: i

Comienzo

pot <- 1

para i desde 1 hasta exponente

pot <- pot * base

Fin

REPETIR exponente VECES
pot <- pot * base

pot := 1;

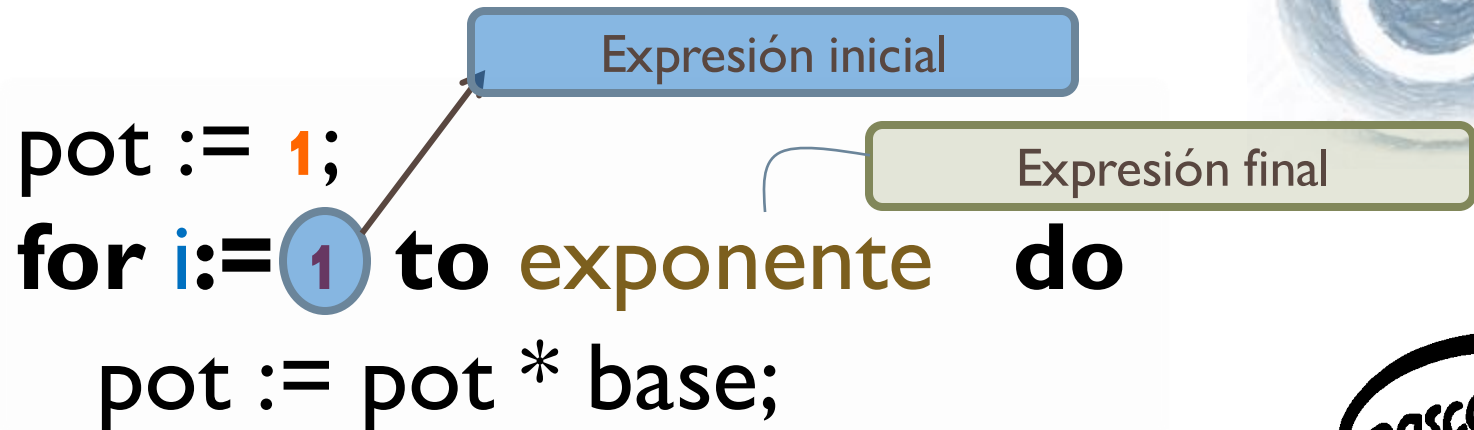
for i:= 1 to exponente do

pot := pot * base;



La **variable de control** toma valores en el rango que determinan la **expresión inicial** y la **expresión final**.

Iteración con contador



- La variable *i* (podemos usar cualquier variable entera) se inicializa al ingresar al *for* con el valor obtenido de evaluar la expresión inicial. Con este valor se ejecuta por primera vez el bucle.
- Automáticamente, *i* se va incrementando en 1 unidad luego de cada ejecución del bucle. La última vez que se ejecuta el *for* es con $i = \text{expresión final}$.

Iteración con contador



```
Program potencia;  
Var pot, i, base, exp: integer;  
begin  
  writeln('Ingrese la base y el exponente');  
  readln(base, exp); //VALIDAR  
  pot := 1;  
  for i:= 1 to exp do  
    pot := pot * base;  
  writeln('El valor de la potencia es:', pot);  
end.
```



Iteración con contador

Program potencia;

Var pot, i, base, exp: integer;

begin

→ writeln('Ingrese la base y el exponente');

readln(base, exp);

pot := 1;

for i:= 1 to exp do

 pot := pot * base;

 writeln('El valor de la potencia es:', pot);

end.

EE

Ingrese la base y el exponente:

base	exp	i	pot

Tabla de datos

Iteración con contador

Program potencia;

Var pot, i, base, exp: integer;

begin

writeln('Ingrese la base y el exponente');

→ readln(base, exp);

pot := 1;

for i:= 1 to exp do

pot := pot * base;

writeln('El valor de la potencia es:', pot);

end.

EE

Ingrese la base y el exponente:
5 3

base	exp	i	pot
5	3		

Iteración con contador

Program potencia;

Var pot, i, base, exp: integer;

begin

writeln('Ingrese la base y el exponente');

readln(base, exp);

→ pot := 1;

for i:= 1 to exp do

pot := pot * base;

writeln('El valor de la potencia es:', pot);

end.

EE

Ingrese la base y el exponente:
5 3

base	exp	i	pot
5	3		1

Iteración con contador

Program potencia;

Var pot, i, base, exp: integer;

begin

writeln('Ingrese la base y el exponente');

readln(base, exp);

pot := 1;

→ for i:= 1 to exp do

pot := pot * base;

writeln('El valor de la potencia es:', pot);

end.

$i \leq \text{exp} \rightarrow 1 \leq 3 \rightarrow \text{TRUE}$

EE

Ingrese la base y el exponente:

5 3

base	exp	i	pot
5	3	1	1

Iteración con contador

```
Program potencia;  
Var pot, i, base, exp: integer;  
begin  
  writeln('Ingrese la base y el exponente');  
  readln(base, exp);  
  pot := 1;  
  for i:= 1 to exp do  
    → pot := pot * base;  
    writeln('El valor de la potencia es:', pot);  
end.
```

$i := i + 1$

$i \leq \text{exp} \rightarrow i \leq 3 \rightarrow \text{TRUE}$

EE

Ingrese la base y el exponente:

5 3

base	exp	i	pot
5	3	1	+
			5

Iteración con contador

Program potencia;

Var pot, i, base, exp: integer;

begin

writeln('Ingrese la base y el exponente');

readln(base, exp);

pot := 1;

→ for i:= 1 to exp do

pot := pot * base;

writeln('El valor de la potencia es:', pot);

end.

i<=exp?

i<=exp → 1<=3 → TRUE

i<=exp → 2<=3 → TRUE

EE

Ingrese la base y el exponente:

5 3

base	exp	i	pot
5	3	+	+
		2	5

Iteración con contador

```
Program potencia;  
Var pot, i, base, exp: integer;  
begin  
  writeln('Ingrese la base y el exponente');  
  readln(base, exp);  
  pot := 1;  
  for i:= 1 to exp do  
    → pot := pot * base;  
    writeln('El valor de la potencia es:', pot);  
end.
```

$i := i + 1$

$i \leq \text{exp} \rightarrow 1 \leq 3 \rightarrow \text{TRUE}$
 $i \leq \text{exp} \rightarrow 2 \leq 3 \rightarrow \text{TRUE}$

EE

Ingrese la base y el exponente:
5 3

base	exp	i	pot
5	3	1	1
		2	5
			25

Iteración con contador

Program potencia;

Var pot, i, base, exp: integer;

begin

writeln('Ingrese la base y el exponente');

readln(base, exp);

pot := 1;

→ for i:= 1 to exp do

pot := pot * base;

writeln('El valor de la potencia es:', pot);

end.

$i \leq \text{exp} \rightarrow 1 \leq 3 \rightarrow \text{TRUE}$

$i \leq \text{exp} \rightarrow 2 \leq 3 \rightarrow \text{TRUE}$

$i \leq \text{exp} \rightarrow 3 \leq 3 \rightarrow \text{TRUE}$

EE

Ingrese la base y el exponente:

5 3

base	exp	i	pot
5	3	1	1
		2	5
		3	25

Iteración con contador

```
Program potencia;  
Var pot, i, base, exp: integer;  
begin  
  writeln('Ingrese la base y el exponente');  
  readln(base, exp);  
  pot := 1;  
  for i:= 1 to exp do  
    → pot := pot * base;  
    writeln('El valor de la potencia es:', pot);  
end.
```

$i \leq \text{exp} \rightarrow 1 \leq 3 \rightarrow \text{TRUE}$
 $i \leq \text{exp} \rightarrow 2 \leq 3 \rightarrow \text{TRUE}$
 $i \leq \text{exp} \rightarrow 3 \leq 3 \rightarrow \text{TRUE}$

EE

Ingrese la base y el exponente:
5 3

base	exp	i	pot
5	3	1	1
		2	5
		3	25
			125

Iteración con contador

Program potencia;

Var pot, i, base, exp: integer;

begin

writeln('Ingrese la base y el exponente');

readln(base, exp);

pot := 1;

→ for i:= 1 to exp do

pot := pot * base;

writeln('El valor de la potencia es:', pot);

end.

i<=exp?

EE

i<=exp → 1<=3 → TRUE

i<=exp → 2<=3 → TRUE

i<=exp → 3<=3 → TRUE

i<=exp → 4<=3 → FALSE

Ingrese la base y el exponente:
5 3

base	exp	i	pot
5	3	1	1
		2	5
		3	25
		4	125

Iteración con contador

```
Program potencia;  
Var pot, i, base, exp: integer;  
begin  
  writeln('Ingrese la base y el exponente');  
  readln(base, exp);  
  pot := 1;  
  for i:= 1 to exp do  
    pot := pot * base;  
  → writeln('El valor de la potencia es:', pot);  
end.
```

Ingrese la base y el exponente:

5 3

El valor de la potencia es: 125

base	exp	i	pot
5	3	1	1
		2	5
		3	25
		4	125

Iteración con contador

```
Program potencia;  
Var pot, i, base, exp: integer;  
begin  
  writeln('Ingrese la base y el exponente');  
  readln(base, exp);  
  pot := 1;  
  for i:= 1 to exp do  
    pot := pot * base;  
  writeln('El valor de la potencia es:', pot);  
end.
```

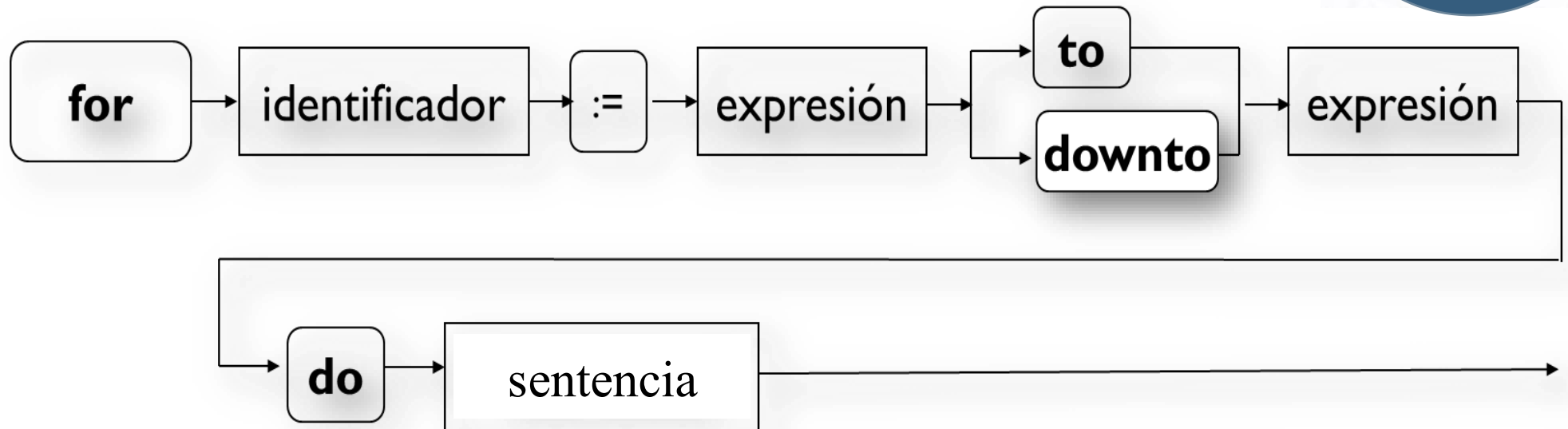
Ingrese la base y el exponente:
5 3
El valor de la potencia es: 125

base	exp	i	pot
5	3	1	1
		2	5
		3	25
		4	125

La variable de control del FOR (i en este caso), durante el FOR, es modificada únicamente por Pascal

Diagrama de SINTAXIS: FOR

Visión
estática



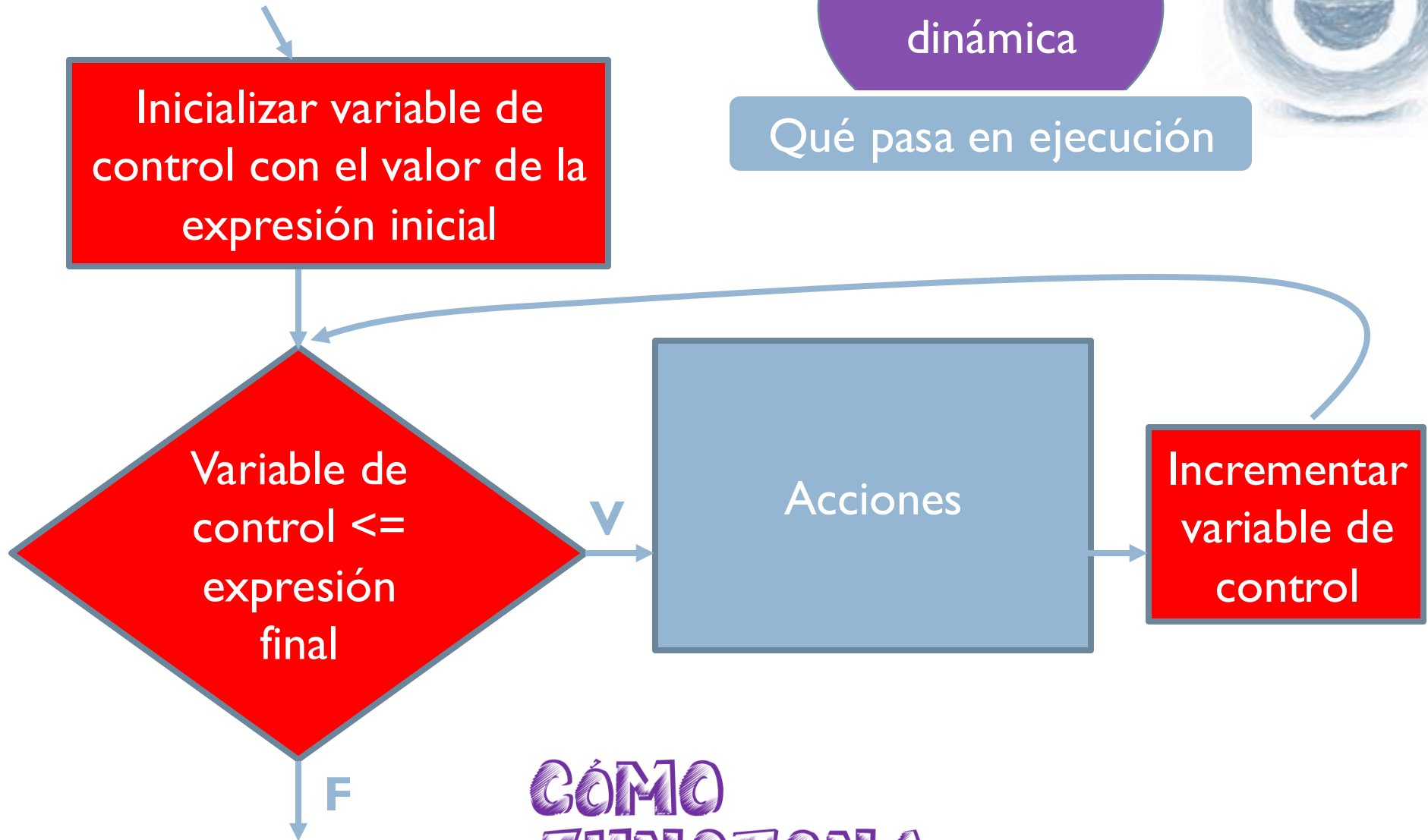
“sentencia” puede ser una o varias acciones
(sentencia simple o compuesta)

CÓMO SE ESCRIBE

Diagrama de flujo: FOR

Visión
dinámica

Qué pasa en ejecución



CÓMO
FUNCIONA

Iteración con contador



Fin FOR



At home



Iteración



Escribir un programa que muestre los factores positivos de un número entero n .

¿ $\boxed{1}$ es divisor de n ?
¿ $\boxed{2}$ es divisor de n ?
¿ $\boxed{3}$ es divisor de n ?

En cada **iteración** se evalúa una **condición**.

¿Cómo lo generalizamos? ¿ $\boxed{i}$ es divisor de n ?

¿Cuándo terminamos?

¿Cómo lo expresamos en Pascal?

Iteración



```
program divisores;  
{Muestra los divisores de un número n}  
var i, n: integer;  
begin  
  write ('Ingrese el valor para n: ');  
  readln(n);  
  writeln ('Los factores positivos de n son ');  
  for i := 1 to n do  
    if (n mod i = 0)  
      then write (i);  
end.
```

El **bloque iterativo** se ejecuta n veces;
el valor de la variable de control es diferente cada vez.

Iteración

Escribir un programa que muestre los factores positivos de un número entero n **de mayor a menor**.

```
program divisores;  
{Muestra los divisores de un número n}  
var i,n:integer;  
begin  
  write ('Ingrese el valor de n');  
  readln(n);  
  writeln ('Los divisores son ');  
  for i := n downto 1 do  
    if (n mod i = 0)  
      then write (i);  
end.
```



Iteración con condición



Problema: Obtener la posición menos significativa en la que aparece un dígito d en un número N .
Si d no está en N , mostrar un cartel indicando esta situación.

Por ejemplo

- Si $d=3$ y $N= 605$ la salida es 0. (una posición 0 indica que no se encontró el dígito).
- Si $d=3$ y $N= 1305$ la salida es 3.
- Si $d=3$ y $N= 36035$ la salida es 2.

Posiciones de un número N con m dígitos: $d_m d_{m-1} \dots d_2 d_1$

Iteración con condición



```
pos:=0; encuentre:=false;  
while ((N>0) and (encontre=false))  
do
```

```
begin  
  pos:=pos+1;  
  if (N mod 10=d)  
    then encuentre:=true  
    else N:=N div 10;  
end;
```

Bloque iterativo

Inicialización de las variables

Condiciones de corte de la Iteración



Iteración con condición

```
program posMenosSig;  
var N, d, pos, Naux: integer; encuentre: boolean);  
begin  
  writeln('Ingrese el número y el dígito a buscar'); readln(N, d);  
  Naux:=N;  
  pos:=0; encuentre:=false;  
  while (N>0 and not encuentre)  
  do  
    begin  
      pos:=pos+1;  
      if (N mod 10=d)  
        then encuentre:=true  
        else N:=N div 10;  
    end;  
    N:=Naux;  
  if encuentre  
    then writeln('El dígito ', d, ' está en la pos ', pos, ' de ', N)  
    else writeln('El dígito ', d, ' no está en ', N);  
end.
```



76

Iteración con contador



Problema: A partir de un natural N , calcular la suma de los primeros N números naturales.

Ej.: Si $N=5$, se desea calcular el resultado de sumar $1+2+3+4+5 \rightarrow 15$

¿Cuáles son los datos de entrada?

¿Cuáles son los datos de salida?

¿Cuáles pueden ser los casos de prueba?

El acumulador comienza vacío (en 0).

Sumar el 1 al acumulador.

Sumar el 2 al acumulador.

Sumar el 3 al acumulador.

Sumar el 4 al acumulador.

...

Sumar el N al acumulador.

**Se repite la acción de sumar
un número al acumulador.
Ese número toma los valores
del 1 hasta N .**

Iteración con contador



Problema: A partir de un natural N , calcular la suma de los primeros N números naturales.

Ej.: Si $N=5$, se desea calcular el resultado de sumar $1+2+3+4+5 \rightarrow 15$

Algoritmo Sumatoria

DE: N

DS: suma

Daux: i

Comienzo

suma $\leftarrow 0$

para i desde 1 hasta N

 suma $\leftarrow$ suma + i

Fin

**i toma los valores
del 1 hasta N .**

Iteración



1. Escribir un programa que, a partir de valores enteros y positivos a y b (con $a < b$) sume los números contenidos en el intervalo $[a, b]$ que sean múltiplos de 3.

Por ejemplo, si $a=10$ y $b=21$, debe devolver el resultado de sumar $12+15+18+21$. Si $a=4$ y $b=5$, debe devolver 0.

2. Escribir un programa que, a partir de un número natural N , sume las apariciones de los dígitos 1 y 5 en N .

Por ejemplo, si $N=54135$, debe devolver 11. Si $N=48$, debe devolver 0.

3. Escribir un programa que, a partir de valores enteros y positivos a y b (con $a < b$) determine si entre los números contenidos en el intervalo $[a, b]$ hay algún múltiplo de 8.

Desafíos: Definir datos de entrada y salida. Buscar una forma de resolución usando la ESTRUCTURA DE CONTROL más conveniente para cada caso (¿iterador con contador o con condición?).

Diagramas Sintácticos

- La sintaxis de un lenguaje de programación es un conjunto de reglas que describen la *forma* del programa.
- Un diagrama sintáctico es una descripción gráfica de la sintaxis de un lenguaje de programación.



Las elipses indican que **texto** se debe incluir tal cual como aparece.



Los rectángulos indican que **nombre** está definido en algún otro diagrama sintáctico.



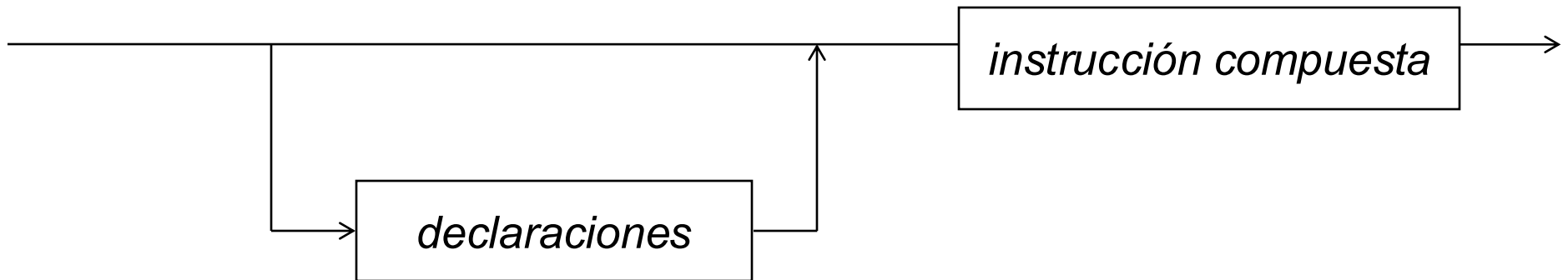
Las flechas indican el orden de lectura en el diagrama.

Diagramas Sintácticos

programa

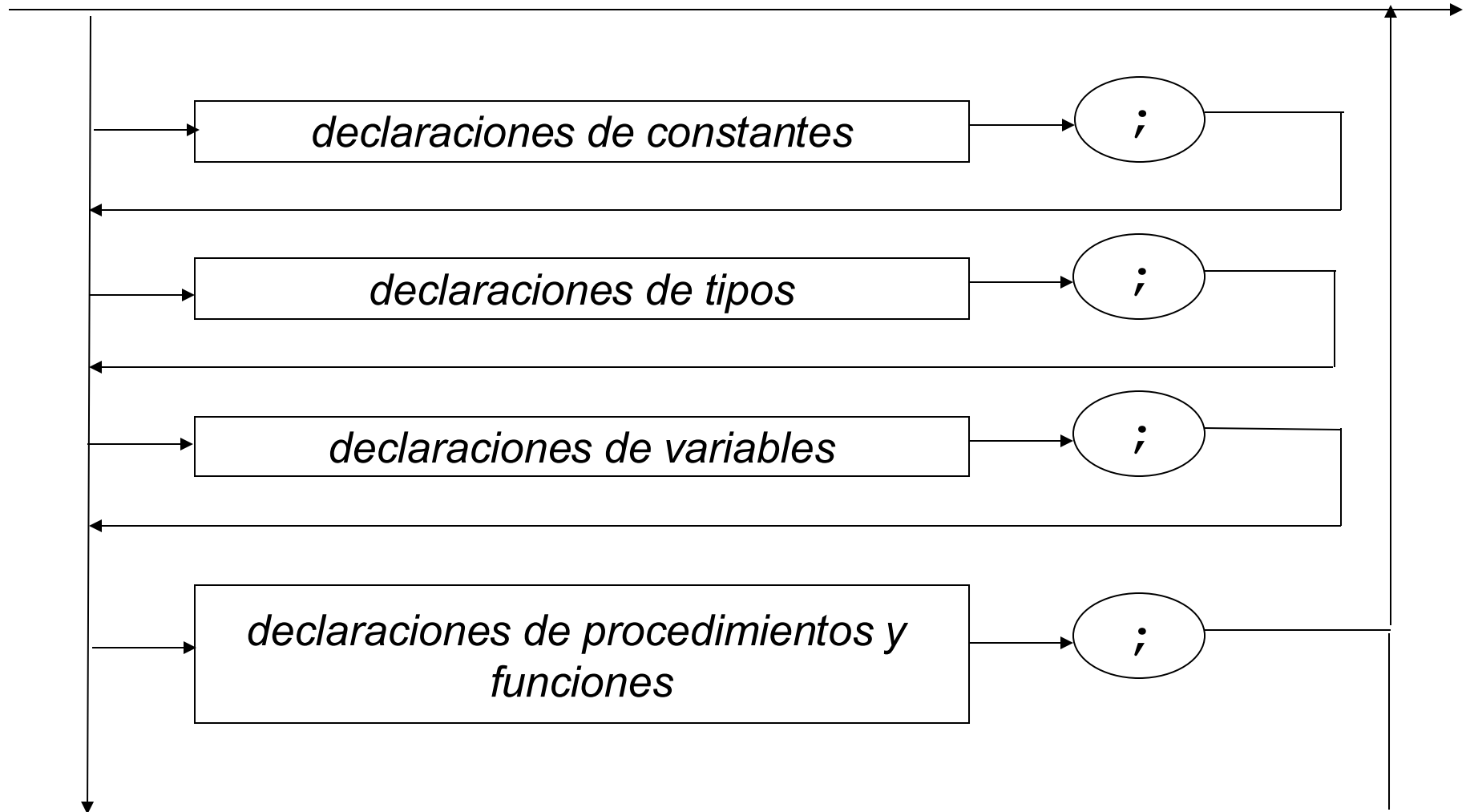


bloque



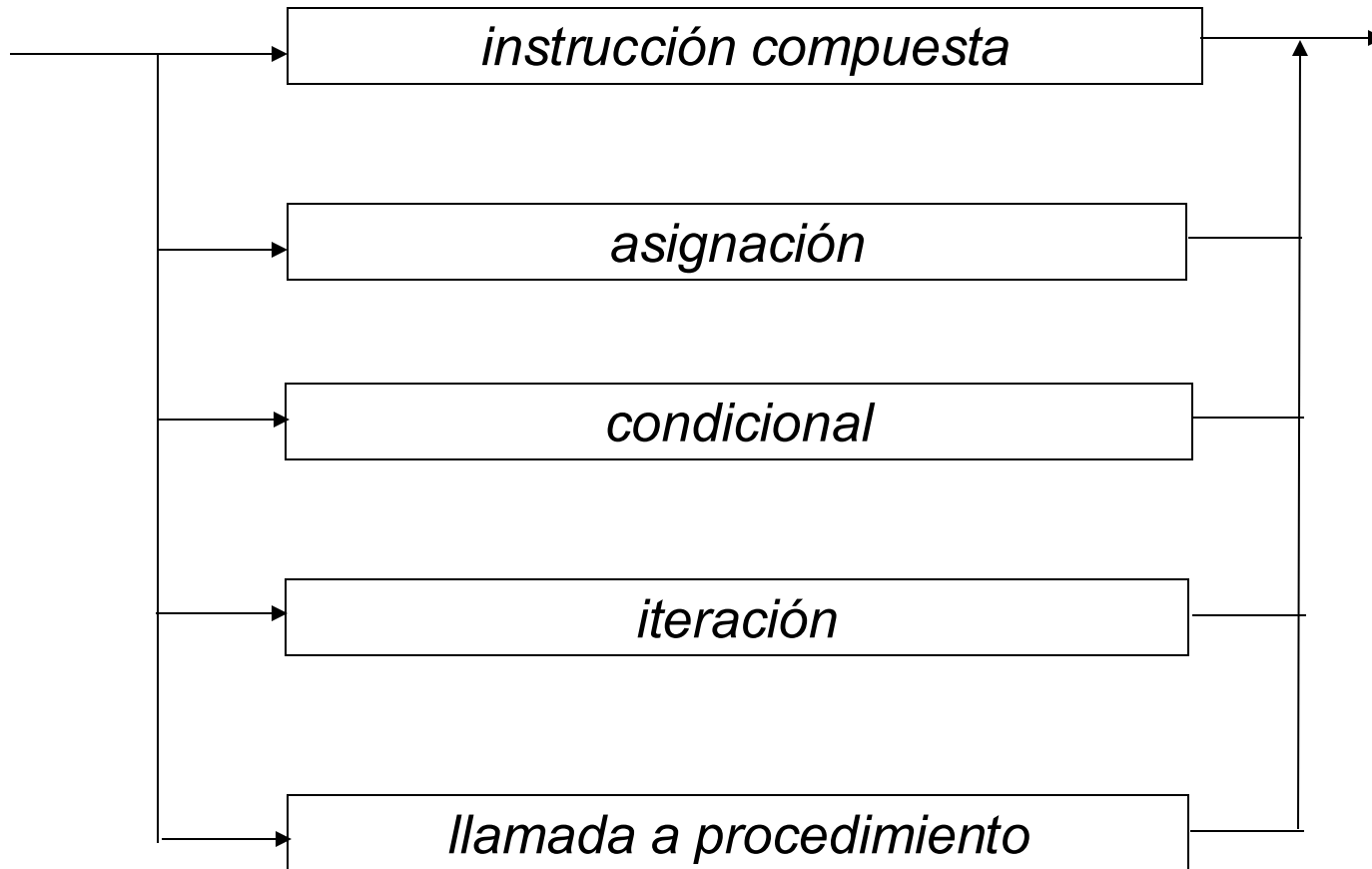
Diagramas Sintácticos

declaraciones



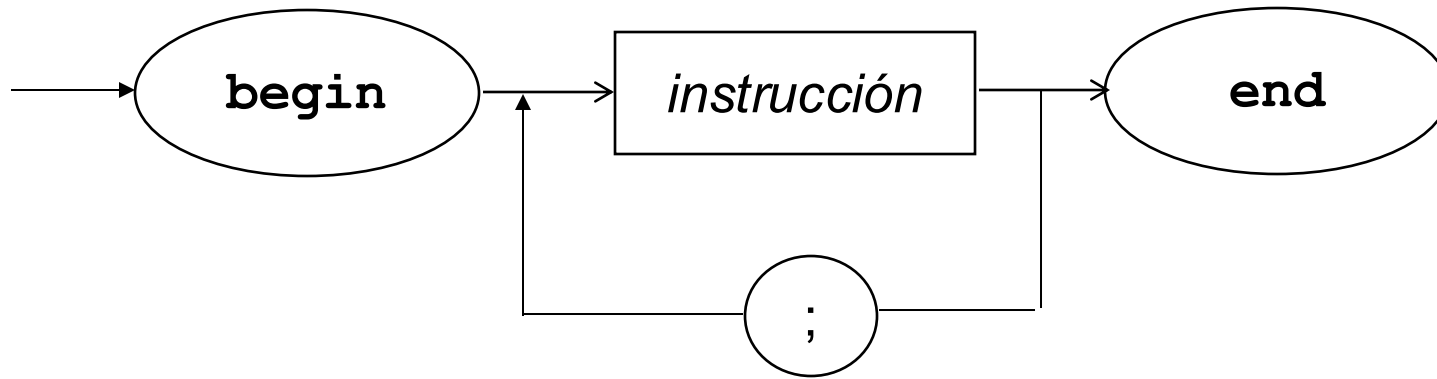
Diagramas Sintácticos

instrucción (también llamada *proposición*)



Diagramas Sintácticos

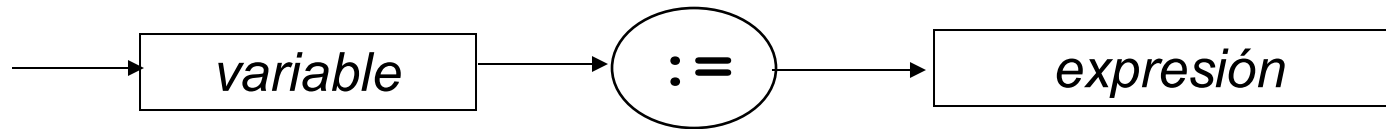
instrucción compuesta



Las instrucciones de una instrucción compuesta se ejecutan en **secuencia**, una a continuación de la otra.

Diagramas Sintácticos

asignación



La **semántica** de la asignación es computar la expresión a la derecha del símbolo $:=$ y luego almacenar el valor computado en la locación de memoria ligada a la variable que aparece a la izquierda.

Para nosotros variable va a ser un identificador declarado como variable.

Diagramas Sintácticos

condicional

